

have successfully completed the basic requirements, download the GlassFish jar file from [download.java.net/javaee5/v2ur2/promoted/Linux/glassfish-installer-v2ur2-b04-linux.jar](http://download.java.net/javaee5/v2ur2/promoted/Linux/glassfish-installer-v2ur2-b04-linux.jar) and issue the following command:

```
java -jar -Xmx=256 glassfish-installer-v2ur2-b04-linux.jar
```

This command will show a licence agreement window—it's dual licensed under CDDL and GPL. After accepting the licence it will create a directory named *glassfish* in your current working directory. After the installation is complete, we need to set it up before being able to use it.

#### Configuration and set-up

To set up GlassFish navigate to the *glassfish* directory from the terminal. Here you will see a file named *setup.xml*. This file is the input file for ant builder scripts that will configure GlassFish. *setup.xml* contains the following configuration property elements:

```
<property name="install.home" value="${basedir}"/>
<property name="domain.name" value="domain1"/>
<property name="instance.name" value="server"/>
<property name="admin.user" value="admin"/>
<property name="admin.password" value="adminadmin"/>
<property name="admin.port" value="4848"/>
<property name="instance.port" value="8080"/>
<property name="orb.port" value="3700"/>
<property name="mq.port" value="7676"/>
<property name="https.port" value="7181"/>
<property name="glassfish.license" value="LICENSE.txt"/>
<property name="glassfish.zipfile" value="./glassfish-image-pe.zip"/>
<property name="glassfish.release.name" value="glassfish"/>
<property name="glassfish.p.release.name" value="glassfish-pe"/>
<property name="glassfish.class" value="glassfish.class"/>
<property name="persistence.location" value="${install.home}/./glassfish.persistence"/>
<property name="glassfish.persistence.zipfile" value="glassfish.persistence-image.zip"/>
<property name="glassfish.persistence.class" value="glassfish.persistence.class"/>
<property name="jarpack-task.jar" value="${install.home}/lib/Pack200Task.jar"/>
<property name="admin.passfile" value="${install.home}/passfile"/>
<property name="asadmin.prefile.file" value="${install.home}/conf/asadminexec.conf"/>
<property name="asadmin.defaultprofile" value="developer"/>
<property name="asadmin.defaultsecure" value="false"/>
```

Before setting up the server, you need to make some changes in the property values according to your preferences. First of all, change the default *admin.password* property, which by default is set as *adminadmin* (see line 5 in the above snippet). You

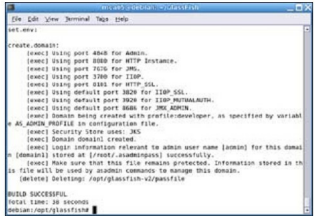


Figure 1: GlassFish build successful

need to remember this password, as it will be used for login and deploying applications later. You can also change the ports on which the admin and applications will listen. Change them accordingly if they conflict with services that listen on the same ports.

After making changes in the configuration file, it's time to start the build process. I will recommend a set-up process such that in the future you can easily update GlassFish with its new releases.

We will move *glassfish* to */opt/glassfish-v2*, and then create a soft-link (*ln -s /opt/glassfish-v2 /opt/glassfish*) such that in the future we can easily upgrade it. Make the ant builder script executable by issuing the following command:

```
chmod u+x /opt/glassfish/lib/ant/bin/ant
```

Now, to start the build process, issue the following command:

```
/opt/glassfish/lib/ant/bin/ant -f setup.xml
```

You should soon get a 'build successful' message as shown in Figure 1.

To start the server and check whether it is installed and configured successfully, issue the following command:

```
/opt/glassfish/bin/asadmin start-domain
```

...and point your browser to <http://localhost:4848>. You should get a login page as shown in Figure 2. If you don't, then recheck the whole procedure for the possible error.

For login, use the password you have applied for the *admin*, *password* property earlier.

## Developing a simple Web application

In this section I will develop a simple dynamic Web application based on the Model View Controller (MVC) design pattern, which will generate prime numbers for its clients. An MVC design pattern divides an application into the following components: